

有限電池 × 變動服務窗口

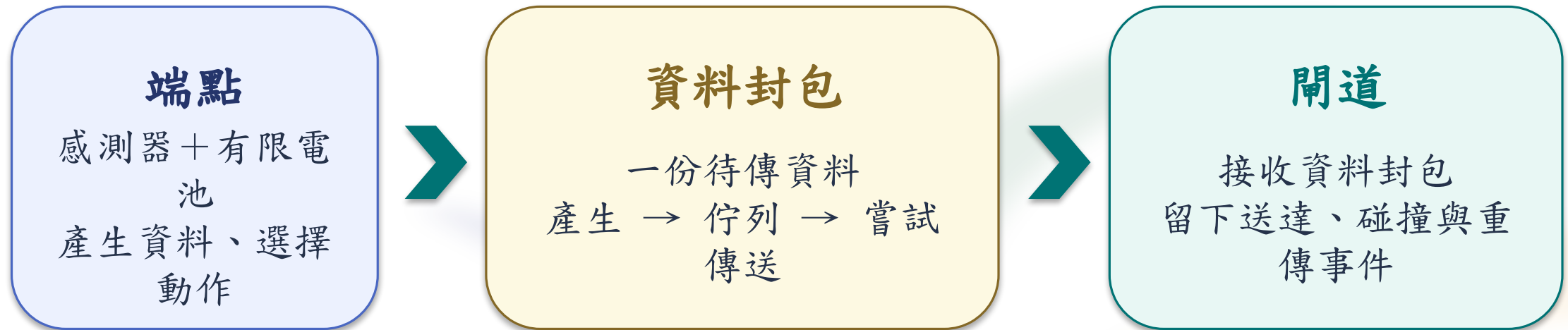


先看窗口，再看動作；最後讀服務與端點能量

scenario (情境定義) | 來源: course package | 作用: 固定端點、窗口與資料規則
單位: identifier | 判讀: 相同輸入可重跑

LoRaEnergySim 是什麼

把端點的一次選擇，放進可重複的封包與能量模擬



它回答：同一份工作，在等待、休眠或傳送的不同決策下，服務與端點能量（J）如何一起變？

simulator（模擬器） | 來源：course package 參照 LoRaEnergySim | 作用：在固定時鐘重播端點、封包與能量假設
單位：一次固定執行 | 判讀：產生可比較證據，不是實測

原始上游套件已做什麼

上游研究模型提供「端點狀態 → 封包事件 → 能量」的可重用能力

上游已有的研究能力

休眠／處理／發射／接收狀態

封包、碰撞、重傳、端點能量

上游沒有替本課固定的部分

固定情境、LEO 服務窗口、案例順序

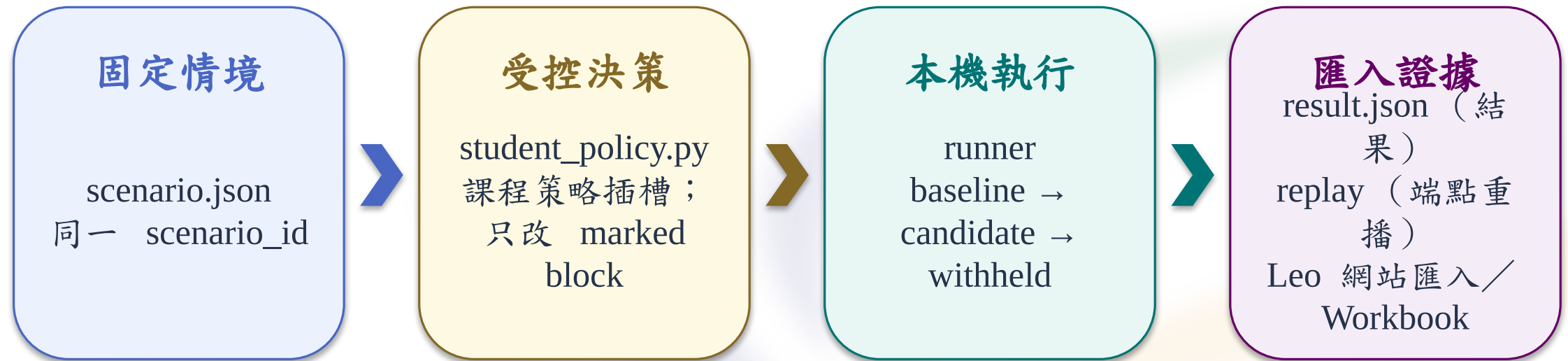
受控策略、結構化匯入、Leo 重播

因此：上游能力是參照；課程流程、案例與證據邊界由本課封裝。

`upstream_model`（上游模型能力） | 來源：GillesC/LoRaEnergySim@f854462c | 作用：提供端點狀態、封包與能量概念
單位：能力集合 | 判讀：僅是上游能力，不是本課 runner 執行

課程封裝補了什麼

把研究模型包成一條可教、可驗、可匯入的固定路徑



student_policy.py 是課程策略插槽； runner 不直接執行上游 LoRaEnergySim，上游僅能力參照。

runner (課程執行器) | 來源：course package | 作用：讀固定 scenario、載入 bounded policy、輸出 result / replay
單位：run artifact | 判讀：把一次修改連到可重播證據

為什麼競賽要用它

先得到可重複的因果證據，再回到 LEO 網站解讀

可重複

同一情境、相同策略檔
與固定種子
重跑同一條事件路徑

可檢驗

改一個受控區塊
檢查封包／服務／狀態
／J 是否改變

可轉移

智慧農業、暖通空調、
物流
重訂窗口、期限與能量
邊界

本套件不是即時資料、典範系統指標或整體 LEO 能量；它先建立可反駁的端點因果證據。

evidence（證據） | 來源：result.json + endpoint replay | 作用：保存 policy → event → service / J 的路徑
單位：record | 判讀：支持 bounded comparison，不擴大 claim

只有 run case 才產生結果檔

setup / verify 只檢查環境；run case 才會寫入一次執行的產物



只有 run case 會產生 artifacts/<run_id>/result.json 與 endpoint-replay.json；setup / verify 不算。

run artifact (執行產物) | 來源：本機 runner | 作用：保存一次執行的 result 與 endpoint replay
單位：同一 run 目錄 | 判讀：只有 run 產生；setup / verify 不算

課程頁 /course 只匯入結果檔

本機 runner 做模擬；網站做驗證、具象化、重播與工作簿

本機 runner (模擬)

Linux / macOS 終端機
WSL 也使用 Linux 指令
讀 student_policy.py
(課程策略插槽)



/course (課程頁) 只選 result.json

驗證

具象化

重播 / 工作簿

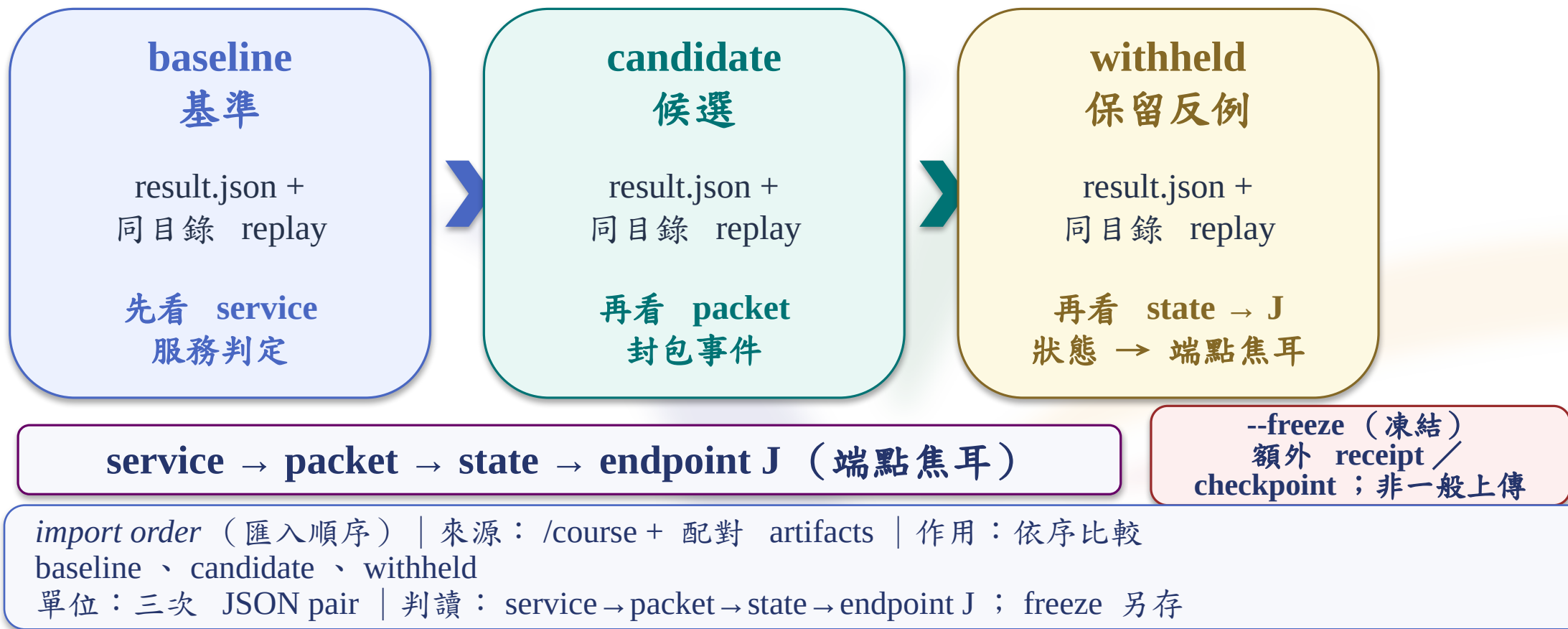
endpoint-replay.json 留在同一目錄配對
不選、不上傳 student_policy.py

policy 原始檔只在本機 runner 讀取；/course 接收 result.json，不接收 Python。

course import (課程匯入) | 來源：/course 控制項 | 作用：驗證 result.json，建立重播與工作簿
單位：JSON pair | 判讀：選結果檔；replay 留同目錄

三次匯入，沿同一解讀順序

baseline → candidate → withheld：每次匯入一對檔案，再按同一順序解讀



一次動作，同時改變服務與能量

同一個 *action* 會沿不同路徑留下可觀察後果

SLEEP | 低功耗休息

休息功率降低
喚醒延遲與能量增加

WAIT | 清醒等待

反應能力保留
清醒閒置能量累積

SEND | 送出封包

服務機會增加
處理與無線事件增加

服務結果

端點能量 (J)

action (策略動作) | 來源: policy API | 作用: 每次決策的唯一輸出
單位: enum | 判讀: 觸發狀態與封包轉移

先讀中間事件，再讀最終結果

能量或服務差異，必須由中間事件支持



窗口關閉 → 傳輸類動作不執行 → 安全分支為 *SLEEP*

若中間事件沒有差異，最後的數值變化不得歸因於本次修改。

radio_state（無線狀態） | 來源：runner event ledger | 作用：表示各狀態區間
單位：state enum + duration | 判讀：提供端點能量的時間分段

固定情境中的四步操作閉環

只改一個受控常數，再沿同一條證據路徑比較

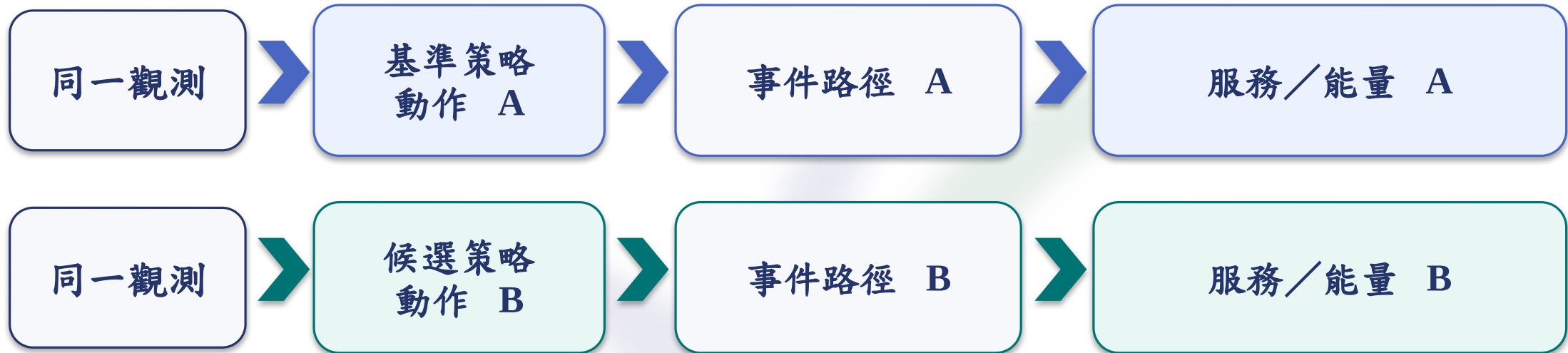


預測與結果同時保存；反例保留為可檢驗證據。

result.json（結果資料檔） | 來源：package runner | 作用：保存 run identity、狀態、封包、服務與能量
單位：JSON artifact | 判讀：作為 replay 與 import 的輸入

中間事件是因果歸因的必要橋樑

同一觀測下，比較基準與候選策略的事件路徑



先確認事件路徑不同，才比較服務與能量。

`packet_progress`（封包進度） | 來源：result / replay artifact | 作用：記錄產生、佇列、嘗試、重試、送達與逾期
單位：event count + time | 判讀：連接策略動作與服務結果

Runner 證據有明確邊界

可重跑，不等於可以外推到所有能量層級

本套件可支持

固定 scenario / seed
端點狀態與封包事件
服務判定與端點 J

本套件不支持

LEO / system energy
canonical efficiency
實測或即時 KPI

endpoint_energy_j (端點能量) | 來源: endpoint radio / processing model | 作用: 累積狀態能量
單位: J | 判讀: 只屬於 endpoint evidence layer

LEO 是變動服務窗口的工作範例

LEO 改變可服務機會；主要控制仍在端點策略



相同機制可以轉移；窗口、期限與能量邊界必須重新定義。



`service_window` (服務窗口) | 來源：fixed scenario trace | 作用：定義合法傳輸區間
單位：trace time | 判讀：決定動作是否具有服務機會

從預測到結果，保留可反駁路徑

先寫下方向，再執行；結果不符時保留反例



保留條件案例維持 frozen policy，用來檢查機制是否可轉移。

prediction（預測） | 來源：workbook record | 作用：指定可反駁的變化方向
單位：statement | 判讀：作為執行前的比較基準

三個實驗到底改什麼

三次都只改 `student_policy.py` 的一個 `marked block` ；
`runner` 、 `scenario` 、 `schema` 不動

Lab A | 休息方式

SLEEP → WAIT
PACE_GAP_STEPS = 2 不變

觀察：SLEEP /
AWAKE_IDLE / WAKE 的時間與 J；封包服務

Lab B | 穩定門檻

STABLE_STEPS : 2 → 1
ENTER_QUALITY=2 /
EXIT_QUALITY=1 不變

觀察：進入時機、attempt /
retry / delivery、service、J

Lab C | 急件提前量

URGENT_MARGIN_S :
20 → 5 → 30
BATCH_SIZE = 3 不變

觀察：urgent
timing、deadline / expired、
service、J

同一檔案 / 同一 `scenario` / 固定 `runner` → `prediction` → `baseline` → `candidate` → `withheld`

marked block（受控區塊） | 來源：`student_policy.py` | 作用：限制可修改的最小決策面
單位：one block | 判讀：其他行與檔案不動，才能歸因